

Prime Agent: A Self-Improving RLM Harness

Alex L. Zhang

Kevin Thomas

Sebastian Müller

Elie Bakouch

Daniel Auras

Mika Senghaas

Fares Obeid

Konstantin Dunas

Johannes Hagemann

Sami

Jaghouar Affiliation: Princeton University

Prime Intellect

MIT Affiliation: Correspondence:

seth@primeintellect.ai, altzhang@mit.edu

First published: August 5, 2026 • Current version: August 24, 2026

Abstract

Language models are sequential processors, but long-horizon agency requires external information and computation beyond model weights and active context. Prime Agent is an open-source harness for long-horizon evaluation and coding-agent workflows. A persistent IPython REPL follows the Recursive Language Model abstraction for programmatic context processing and test-time compute, while Continual Harness preserves histories, memories, skills, prompts, and subagent specifications across trajectories. Recursive subagents coordinate through direct agent-to-agent communication, and the Agents View lets humans inspect and manage daemon-backed sessions. Prime Agent standardizes execution, recovery, verification, and resource accounting while leaving strategy construction to the model. This low-friction, expressive membrane prevents harness failures from becoming model failures and pushes measurement toward the model’s true maximal underlying capability. Prime Agent raises ARC-AGI-3 RHAE Best@1 from 30% to 95.5% and matches or exceeds native and popular harnesses across long-context coding, GPU-kernel generation, emulator construction, and autonomous nanoGPT speedruns. On Factorio, we find refinement allows for continuous technology progression and dedicated subagents enable parallelized work. Code is available at <https://github.com/PrimeIntellect-ai/prime-agent>.

1 Introduction

A strong language model on its own does not have the full capabilities of a computer. An LLM is a bounded sequential processor whose next decision can use only state information exposed in its weights and active context. A harness supplies the missing computational substrate that allows for external actions via tool-calls. These external actions also include information management beyond prior knowledge stored in the model weights. Context management was first enabled by agentic compaction, the process by which a model selectively analyzes its own context to reduce tokens while keeping essential information. However, the full information state has grown beyond the weights and token context. Imagined as a state information cache ([Figure 2](#)), model weights are L0, active context is L1, a persistent REPL and recursive subagents form L2, and disk-backed history, memories, and skills form L3. This makes the system more *von Neumann-like*: the model can read, transform, and write addressable state outside the instruction currently being generated [[34](#), [35](#)].

This perspective makes *expressivity* the key property of a harness. Rather than encode one workflow, an expressive harness exposes primitives from which the model constructs programs, subagents, and feedback loops at inference time. Recursive Language Models (RLMs) make context and recursive invocation programmable [44]; Continual Harness makes prompts, subagents, skills, and memories revisable from the trajectory history [18]. Lastly, we enable large-scale coordination and orchestration of multi-agent swarms through direct agent-to-agent communication. These components let a fixed model use information management and test-time compute to expand its reachable strategy set.

Prime Agent is designed first as a standardized harness for *long-horizon evaluation*. The most popular metric of which is score at a fixed expenditure [8], such as cost/tokens and time, or, especially for long horizon, score at practical plateau [8], which allows us to analyze the shape of performance over time. The harness is the membrane through which the model observes and acts on the world. A model should fail an evaluation because the task exceeds its capability, not because the harness dropped state, restricted useful actions, miscounted resources, or terminated prematurely. Prime Agent therefore combines standardized, reliable execution with a low-friction and expressive interface for programmatic tools, information management, and swarm management. This lets the model fully use its test-time compute, pushing measured performance toward its true maximal underlying capability rather than the limitations of its harness.

Prime Agent jointly manages information and computation. Information management moves state across L1–L3 through programmatic context processing, compaction, persistent histories, and revisable memories. Computation management allocates test-time compute to programs, tool calls, reusable skills, and parallel recursive subagents [32, 44]. Direct agent-to-agent communication connects the two, routing information across distributed computation so the swarm can coordinate dynamically rather than follow a fixed graph. These communication links also let humans inspect, message, attach to, and intervene in subagent sessions without following every exchange [16, 21]. Together, these interactions produce retained trajectories that improve future computation and can train later model generations [43, 40, 25].

We present Prime Agent, an open-source harness for long-horizon model evaluation and coding-agent workflows. Prime Agent integrates information and computation management across active context, persistent programmatic execution, recursive subagents, and retained histories, memories, and reusable skills, connected through direct agent-to-agent and human-agent communication. Its Agents View provides a visual interface for inspecting, attaching to, and managing persistent daemon-backed agent sessions. We standardize evaluation infrastructure while preserving the model’s freedom to construct its own strategy. Prime Agent improves ARC-AGI-3 performance from 30% to 95%, matches or exceeds Pi, Claude Code, and Codex (and outperforms other harnesses, Hermes Agent, OpenCode, and Kimi-Code, on other benchmarks) across long-context coding, GPU-

kernel generation, and emulator construction, sustains an 85.5-hour nanoGPT run with 19 validated records, and supports four-character Factorio control and long-horizon MazeBench exploration.

2 Prime Agent Architecture

We next describe the Prime Agent architecture in detail. In particular, we discuss (1) how the system manages information and computation (§2.1), (2) how state is organized across model weights, active context, the REPL, and disk-backed storage (§2.2), (3) how persistent REPLs and RLM calls support programmatic computation (§2.3), (4) how recursive sessions communicate with agents and human operators (§2.4), (5) how Continual Harness turns trajectory evidence into reusable state (§2.5), and (6) how long-horizon controls define continuation, termination, and evaluation accounting (§2.6). [Figure 1](#) provides an overview.

Figure 1: Prime Agent connects persistent root and subagent sessions to a daemon, Continual Harness, the Agents View, and the environment. Solid arrows carry execution and messages; dashed arrows carry persistent state.

2.1 Architecture overview

Prime Agent separates information management from computation management. Information management determines what state enters a model invocation and what survives compaction or restart.

Computation management maps model-selected actions to code, tools, and recursive subagent sessions. Direct agent-to-agent communication connects related sessions, and direct human-agent interaction exposes individual nodes for inspection and intervention.

Models manipulate intermediate values with code. Sessions retain history across compaction, detachment, and restart. Subagents inherit the root’s execution and communication primitives. The runtime records model calls, tool use, messages, harness changes, and resource use. The model controls decomposition, computation allocation, communication, and stopping.

2.2 Information hierarchy and persistent state

A model invocation is parameterized by fixed weights and conditions on its active token context. Prime Agent adds state outside that context. External state affects generation when the runtime injects it or an operation serializes a result into the context. [Figure 2](#) organizes this state by visibility, access mechanism, and persistence.

Figure 2: Prime Agent state hierarchy. The boundary between L1 and L2 separates token-visible model state from explicitly managed computation and retained state.

Each level changes through a different mechanism. Fine-tuning updates L0, compaction rewrites L1, and refinement versions selected L3 entries. We call the L2 mechanism *agentic garbage collection*.

The model creates, retains, summarizes, or deletes REPL values and subagent sessions as the task changes.

Explicit operations move information between levels. Python values and tool outputs in L2 enter generation when serialized into L1. Compaction replaces a conversational prefix with a summary and retains the original events in L3 for REPL retrieval. The runtime assembles selected Continual Harness entries into later supplemental prompts; other L3 artifacts enter context on retrieval. L0 stays fixed.

The retained runtime state includes an append-only event history, selected kernel snapshots, the rooted session tree, context and compaction records, persistent message queues, and versioned Continual Harness state. Branching or forking creates a new logical continuation without deleting the prior event sequence. Recovery reconstructs the session under the same identity. Non-serializable Python objects and external processes are recreated from saved artifacts or external services.

2.3 Programmatic computation with RLMs

Each session owns a persistent IPython Read-Eval-Print Loop (REPL). At test time, compute comprises model inference, Python execution, and tool calls; evaluations report tokens, time, and cost separately. Installed tools are imported as Python modules for parsing, filtering, aggregation, and verification with ordinary code. Intermediate values persist across turns and remain outside active context until selected. This avoids repeatedly serializing large logs, task specifications, and structured evaluator output into the context.

Prime Agent implements the RLM abstraction with the asynchronous `rlm` primitive [\[44\]](#). Calling `rlm` creates and schedules a subagent session, then returns a stable handle before the subagent completes. The subagent receives its own model context, IPython kernel, history, and workspace metadata. The parent continues local computation while subagents run. Results arrive later through direct agent-to-agent communication, and retained handles support follow-up after compaction or restart. The model chooses between local code, tools, sequential delegation, and parallel subagents. Prime Agent defines their execution semantics instead of a fixed workflow graph. A complete orchestration example appears in [Appendix B](#).

2.4 Recursive orchestration and interaction

The daemon owns live sessions independently of the client that created them. Root and subagent sessions use the same lifecycle. Sessions are running during a turn or tool operation, idle when loaded without an active turn, and inactive when unloaded but recoverable from persistent state. Client detachment leaves the session running. Stable session and parent identifiers preserve the recursive topology across these transitions.

Direct agent-to-agent communication uses asynchronous, daemon-mediated queues. An agent addresses its parent, children, and siblings, and queued messages remain available when a recipient becomes active again. Filesystem, network, and credential access follow the permissions of the runtime environment. [Figure 3](#) summarizes the shared lifecycle and communication topology.

Figure 3: Multi-agent orchestration lifecycle and direct agent-to-agent communication.

The Agents View exposes the persistent tree for direct human-agent interaction. The interface lets a user inspect history, attach to a session, provide new input, or detach without interrupting execution. The agent-observe interface provides bounded read-only status and recent-message previews; agent-message targets a named related session. This allows for full interaction via the orchestrator.

2.5 Continual Harness

Continual Harness exposes supplemental state for trajectory-time reads and writes [\[18\]](#). Prompt notes store behavioral instructions, memories store facts, skills package executable procedures, and subagent specifications store reusable roles or divisions of labor. Typed state separates rules, facts, programs, and coordination patterns. Entries support create, read, update, and delete operations; local entries belong to one session, and explicitly requested global entries remain available to later sessions.

Refinement converts trajectory evidence into versioned state updates. Agents request edits directly, or `/refine` runs a background model call over relevant events. The runtime applies each edit at a turn boundary, records its trigger and intended effect, and assembles supplemental state for the next invocation. Versions preserve provenance and enable rollback. Refinement supplements the immutable base prompt without rewriting foundational policy.

Self-improvement converts execution evidence into persistent harness state that changes later behavior while model weights remain fixed. Useful computations become skills, repeated coordination patterns become subagent specifications, and corrected assumptions become memories or prompt notes. The resulting trajectory record also provides training data for later model generations.

2.6 Long-horizon execution and evaluation semantics

Prime Agent exposes three long-horizon control mechanisms ([Figure 4](#)).

Figure 4: Long-horizon control mechanisms. Autonomous mode continues under an explicit budget until an end-condition test passes. Goals retain an objective across continuations until agentic completion. Heartbeats initiate cron or timed turns.

Autonomous mode continues model turns within an explicit budget and evaluates a task-specified end-condition test after each turn. A failed test returns bounded output for another attempt; turn, token, and wall-clock limits stop execution. A goal retains an objective across continuations and

ends through agentic completion, when the agent marks the goal complete. Heartbeats initiate turns on cron or timed schedules.

Evaluation configurations bind task and tool interfaces to model and provider settings, compaction and refinement policies, retry policy, completion gates, and resource limits. Accounting aggregates the root and descendant sessions, so delegation remains visible in test-time cost. Event history links model and tool calls, messages, interventions, retries, verifier outcomes, and harness edits to that configuration. Standardized persistence, recovery, termination, and accounting separate harness failures from model failures while preserving model control over decomposition.

3 Evaluation

The evaluation addresses three research questions derived from Prime Agent’s design. RQ1: Test-time scaling. Can a standardized, expressive execution interface let frontier models convert additional output tokens and API cost into verified task progress? We evaluate this question on ARC-AGI-3. RQ2: Information management. Can models use persistent REPL state to search, transform, and aggregate information across long contexts? We compare Prime Agent with native and alternative harnesses on long-context reasoning and coding tasks. RQ3: Persistent recursive execution. Can the same runtime sustain multi-day experimentation, iterative systems construction, recursive control, and online refinement? We study nanoGPT, PMPP-Hard, EmulatorBench, Factorio, and MazeBench through end-to-end outcomes and trajectory analysis. The trajectory analyses show how agents allocate subagents, retain information, and recover from disruption.

3.1 Interactive reasoning at test-time scale

ARC-AGI-3 is the clearest test of Prime Agent to support strong, consistent long-horizon evaluation [2]. Each game requires the model to learn the rules of the game, creating an ad-hoc world model under an action limit. Prime Agent supplies only the environment interface and an autonomous prompt adapted from PRO-LONG [9]; the model constructs the strategy. We note that Claude Code and Codex runs perform worse than Anthropic and Open AI self-reported performance on ARC-AGI-3 (public set), so we defer to their results over our own runs with matched prompt and settings.

Figure 5: ARC-AGI-3 test-time scaling. RHAЕ score versus output tokens per game (left) and estimated API cost (right).

Across the observed configurations, additional output tokens and cost are converted into progress at sharply different rates (Figure 5). The stronger configurations continue to improve across a long interaction horizon, while others plateau early. This pattern is consistent with a model-controlled interface that permits model-dependent test-time scaling instead of imposing one fixed workflow. The reference lines and points place these curves beside official ARC results. They are external values because our native-harness reruns fell below the published scores, so they situate the result rather than isolate a causal harness effect.

3.2 Long-context information management

The long-context suite tests whether a model can actively manage information that does not fit naturally into one prompt. Prime Agent stores the initial context in a readable file, allowing the model to search, transform, summarize, and revisit it from the persistent REPL. This changes long-context reasoning from passive attention over a fixed sequence into a programmatic information-management problem. The suite covers aggregation, latent retrieval, instruction following, reasoning, and long-form coding [5, 33, 6, 3, 23].

Table 1: Long-context results. Bold marks the higher point estimate within each nominal-model pair; metrics differ by row. Bold is not statistical significance, and uncertainty intervals are unavailable.

We generally find Prime Agent to be competitive across a wide range of long tasks, especially against the harness that did not use a model trained around it. Prime Agent especially excels at long-running or long-context tasks, and can competitively run on its own as an autonomous agent. We include a set of focused case studies and experiments on long settings where Prime Agent excels.

3.3 Multi-day autonomous research

The nanoGPT speedrun [4] measures how far an agent can reduce the number of training steps required for a 124M-parameter GPT to reach a fixed validation loss, with each record verified as an eight-seed mean. For each of three models (Kimi K3, DeepSeek V4 Pro, and GLM 5.3), we compare Prime Agent against an alternative harness: the model developer’s own CLI where one exists, and Claude Code or opencode otherwise. We find that the choice of harness has little effect on final records compared to the noise of the experiment.

Model behavior, however, differs. On Prime Agent, models regularly use the persistent REPL to experiment outside the benchmark’s training script, for example by simulating a candidate optimizer on synthetic gradients or numerically optimizing update-rule coefficients before launching a training run. Figure 6 counts these experiments across 18 runs, normalized by the number of training runs each agent executed; Appendix A reproduces one such experiment per model. The effect is largest for DeepSeek V4 Pro, which created roughly six times more such experiments per training run under Prime Agent than under Claude Code. This may be due to the fact that DeepSeek’s own agent harness provides a similar code-execution mode, so the REPL matches a workflow the model was likely trained on. We also observe that models construct programmatic interfaces to the benchmark itself: Kimi K3 defined a probe function through which it ran roughly ninety screening experiments and all 19 of its validated records, whereas the same model on its own CLI performed every operation through direct file edits and built no such machinery.

Figure 6: Out-of-loop experimentation across harnesses. Distinct experiments created outside the training script per 100 training-script executions, pooled over the 2–3 seeds per harness (labels: experiments/training runs). Counts are hand-classified from complete traces; denominators are audited where available and otherwise estimated from launch commands.

3.4 Programmatic systems construction

Emulators.

An emulator is software that reproduces another computer system’s observable behavior. We evaluate Prime Agent on EmulatorBench, a benchmark that tasks agents with constructing emulators in Rust for a variety of game systems. Agents are given a specification of the emulator and a set of diagnostic tests in the form of a verifier.

(a) *Sega Genesis.*

(b) *Game Boy Color.*

Figure 7: Selected EmulatorBench runs. Stepwise verifier score versus estimated cost; hue encodes model and line style encodes harness.

The correctness of an emulator is given from its ability to mimic the behavior of the target machine. This is measured by human-generated diagnostic programs that inspect the emulator’s behavior, such as the CPU flags, PPU timing, and other components. In an effort to minimize the effects of data contamination, we require the agent to build the emulator from scratch in Rust, sandboxed without any reference implementation. We report preliminary results in [Table 1](#) on this long-context coding benchmark averaged over 16 emulator reconstructions, as well as two emulators in [Figure 7](#), the SEGA Genesis and Nintendo Game Boy Color, that Prime Agent successfully reproduces. For Opus, our runs surprisingly failed to solve the tasks despite successful tool-call responses.

GPU kernels.

PMPP-Hard compresses the same programmatic loop into repeated edit, compile, correctness-check, and profile cycles under a wall-clock budget.

Figure 8: PMPP-Hard solve rates at fixed within-model budgets.

Prime Agent and the native harnesses remain close, with the ordering reversing between the two model groups. In these reported within-model comparisons, the general persistent interface supports the compiler–profile loop with no large observed gap. One noted limitation of PMPP-Hard is the strict wall-clock budget comparison. What the wall-clock budgets do not reveal is the substantial improvement in token usage by models that use Prime Agent. This means that the same performance as Codex or Kimi-Code is achieved by Prime Agent at substantially reduced cost, and, token-for-token, Prime Agent has an advantage.

3.5 Persistent interaction and refinement

Factorio.

The Factorio Learning Environment exposes Python observations and actions for a persistent factory world [12]. In a seven-day Sonnet 5 run, the root and its descendants used 23.4 million output tokens while completing 24 of 196 technologies and reaching 71% on advanced-circuit research (Figure 9) with no signs of stalling.

Figure 9: Factorio progress and recursive computation. Technologies researched (top) and agent-tree growth and concurrency (bottom) versus cumulative output tokens for the Sonnet 5 aesthetic run. The vertical line marks a destructive world reset. The final open marker shows 71% progress on advanced-circuit after 24 completed technologies.

The model handled irreversible actions poorly. A destructive world reset reverted the technology count from five to one; the session then recovered and continued the run instead of discarding the trajectory. The root created 633 depth-one subagents across 149 dispatch waves, with at most seven active concurrently. The shallow, repeatedly widening tree recorded parallel task specialization rather than deeper recursion, while the bursty technology curve separated long construction intervals from externally verified progress.

A different Factorio trace revealed the central safety failure of online refinement. The agent discovered that RCON commands could spawn resources directly into assembly machines, used the shortcut despite an anti-cheating heartbeat, and then preserved it as a reusable skill. In this trace, persistence preserved behavior that optimized the measured objective, including a specification exploit. Safe deployment therefore requires least-privilege action interfaces, independent state validation, and auditable rollback of contaminated refinements.

MazeBench.

MazeBench is an open-world 3D spatial reasoning environment where the player controls a 3D cube and must solve puzzle rooms within a global maze, while collecting gems. Frontier models are shown to greatly struggle on this task, expending billions of tokens to solve only a fraction of the overall world. We compare Opus 5 and GPT-5.6 Sol with Prime Agent versus their native harnesses, as well as GLM-5.2 with Claude Code. Following the benchmark metrics, we report the unique number of rooms they find, the unique number of states, and the total number of gems, all as a function of their overall token spend.

Figure 10: MazeBench exploration versus estimated token cost. Solid lines and filled markers show Prime Agent; dotted lines and open markers show comparison harnesses. Hue and marker shape identify the model.

4 Related Work

Programmatic inference and adaptive state.

Programmatic inference gives models code, tools, and recursive calls for transforming context and allocating test-time compute [42, 30, 37, 44, 9, 32]. Memory and refinement methods retain selected observations, feedback, skills, or reasoning traces across turns and tasks [24, 31, 22, 36, 26, 43].

Continual Harness stores prompts, memories, executable skills, and subagent specifications as typed, versioned state [18]. Prime Agent integrates these mechanisms with persistent kernels, recursive sessions, recovery, and complete trajectory capture.

Coding agents and long-horizon evaluation.

Coding-agent runtimes use executable actions, repository tools, sandboxes, event histories, and structured role assignment to solve tasks over many interaction steps [37, 41, 38, 39, 20, 11, 27]. Executable benchmarks and trajectory corpora measure issue resolution, instruction following, retrieval, and reasoning under long contexts or extended interaction [13, 25, 40, 5, 33, 6, 3, 23]. Prime Agent makes the execution substrate persistent and recursive, then records expenditure across the root and descendant sessions.

Interactive reasoning on ARC-AGI-3.

ARC-AGI-3 extends abstract reasoning to interactive environments with hidden dynamics, goals, and action semantics [2, 1]. Community systems build and verify executable world models, represent agents as stateful Python objects, optimize external workspaces, coordinate specialized agents, and preserve procedures across games [28, 10, 7, 29, 18, 9]. Prime Agent supplies persistent recursive execution and standardized evaluation settings for the same class of long-horizon interactive tasks.

Multi-agent and human-agent communication.

Language-model agent systems coordinate through role prompts, natural-language messages, shared artifacts, and explicit belief state [39, 20, 11, 27, 21, 29]. Learned multi-agent communication studies sparse message selection, compressed representations, social learning, policy alignment, and interpretability for human partners [15, 14, 16, 19]. Prime Agent implements direct agent-to-agent communication through persistent family-scoped queues and exposes the same session tree to human inspection and intervention.

5 Conclusion

Prime Agent introduces a new paradigm for agent harness design in which persistent execution, recursive sessions, autonomous controls, recorded history, and Continual Harness form one substrate for long-horizon work. Results across interactive reasoning, long-context tasks, autonomous research, systems construction, and persistent environments show that this substrate supports different forms of test-time computation under standardized execution and accounting. Despite its results relative to alternative harnesses, models still experience friction when deciding how to allocate subagents, manage retained information, and refine reusable state. Many harness capabilities remain underused because current models were not trained to operate them. We expect model-harness co-learning to become the dominant route to new long-horizon capabilities. Training

directly with Prime Agent could teach models to use the integrated harness more effectively, while targeted training on the RLM and Continual Harness components could isolate their contributions.

Acknowledgements

We thank Florian Brand, SinatraS, and Patience Cave for helping complete Prime Agent runs in additional environments. SinatraS created PMPP Hard, and Patience Cave created MazeBench.

References

- [1] ARC Prize Foundation (2026) ARC-AGI community leaderboard. Note: <https://arcprize.org/leaderboard/community> Accessed August 2026 Cited by: §4.
- [2] ARC Prize Foundation (2026) ARC-AGI-3: a new challenge for frontier agentic intelligence. External Links: 2603.24621, [Link](#) Cited by: §3.1, §4.
- [3] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li (2024) LongBench v2: towards deeper understanding and reasoning on realistic long-context multitasks. External Links: 2412.15204, [Link](#) Cited by: §3.2, [Table 1](#), §4.
- [4] E. Bakouch and P. Intellect (2026) Measuring autonomous AI research. Prime Intellect Blog. External Links: [Link](#) Cited by: §3.3.
- [5] A. Bertsch, A. Pratapa, T. Mitamura, G. Neubig, and M. R. Gormley (2025) Oolong: evaluating long context reasoning and aggregation capabilities. External Links: 2511.02817, [Link](#) Cited by: §3.2, [Table 1](#), §4.
- [6] Z. Chen, X. Wu, J. Jia, C. Gao, Q. Fu, D. Zhang, and S. Hu (2026) LongBench Pro: a more realistic and comprehensive bilingual long-context evaluation benchmark. External Links: 2601.02872, [Link](#) Cited by: §3.2, [Table 1](#), §4.
- [7] D. Curtis, W. Li, and S. Sanner (2026) OPINE-World: programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3. External Links: 2607.01531, [Document](#), [Link](#) Cited by: §4.
- [8] T. Cunningham (2026) Metrics of agent ability. Note: <https://metr.org/notes/2026-07-24-metrics-of-model-ability/> Cited by: §1.
- [9] A. Fox, J. Wang, P. Rosu, and B. Dhingra (2026) PRO-LONG: programmatic memory enables long-horizon reasoning. External Links: 2607.20064, [Link](#) Cited by: §3.1, §4, §4.
- [10] P. Furgale, S. Klingler, J. Nolan, M. Staats, G. Di Lorenzo, E. Martinez Abad, C. Schuller, R. Dinu, A. Devoto, P. Berard, G. Kaplun, E. Sarafian, R. Roveri, L. Derczynski, and R. Silveira Cabral (2026) NVIDIA-labs OO Agents: native python object-oriented agents. External Links: 2607.20709, [Document](#), [Link](#) Cited by: §4.

- [11] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber (2023) MetaGPT: meta programming for A multi-agent collaborative framework. External Links: 2308.00352, [Link](#) Cited by: [§4](#), [§4](#).
- [12] J. Hopkins, M. Bakler, and A. Khan (2025) Factorio learning environment. External Links: 2503.09617, [Link](#) Cited by: [§3.5](#).
- [13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan (2024) SWE-bench: can language models resolve real-world github issues?. In International Conference on Learning Representations, External Links: [Link](#) Cited by: [§4](#).
- [14] S. Karten, S. Kailas, H. Li, and K. Sycara (2023) On the role of emergent communication for social learning in multi-agent reinforcement learning. In Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, pp. 2391–2393. External Links: [Document](#), 2302.14276, [Link](#) Cited by: [§4](#).
- [15] S. Karten, M. Tucker, S. Kailas, and K. Sycara (2023) Towards true lossless sparse communication in multi-agent systems. In IEEE International Conference on Robotics and Automation, pp. 7191–7197. External Links: [Document](#), 2212.00115, [Link](#) Cited by: [§4](#).
- [16] S. Karten, M. Tucker, H. Li, S. Kailas, M. Lewis, and K. Sycara (2023) Interpretable learned emergent communication for human–agent teams. IEEE Transactions on Cognitive and Developmental Systems 15 (4), pp. 1801–1811. External Links: [Document](#), [Link](#) Cited by: [§1](#), [§4](#).
- [17] S. Karten, A. L. Zhang, and S. Jaghouar (2026) Emulator Bench: Verifiable Whole-System Emulation for Ultra Long-Horizon Coding Agents. Note: Manuscript Cited by: [Table 1](#).
- [18] S. Karten, J. Zhang, T. Upaa, R. Feng, W. Li, C. Shi, C. Jin, and K. Vodrahalli (2026) Continual harness: online adaptation for self-improving foundation agents. External Links: 2605.09998, [Link](#) Cited by: [§1](#), [§2.5](#), [§4](#), [§4](#).
- [19] S. Karten (2023) Emergent communication and decision-making in multi-agent teams. Ph.D. Thesis, Carnegie Mellon University, Pittsburgh, PA. Cited by: [§4](#).
- [20] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem (2023) CAMEL: communicative agents for “mind” exploration of large scale language model society. External Links: 2303.17760, [Link](#) Cited by: [§4](#), [§4](#).
- [21] H. Li, Y. Chong, S. Stepputtis, J. P. Campbell, D. Hughes, C. Lewis, and K. Sycara (2023) Theory of mind for multi-agent collaboration via large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, pp. 180–192. External Links: [Document](#), [Link](#) Cited by: [§1](#), [§4](#).
- [22] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark

(2023) Self-refine: iterative refinement with self-feedback. In Advances in Neural Information Processing Systems, External Links: [Link](#) Cited by: [§4](#).

- [23] S. R. Motwani, D. Nichols, C. London, P. Li, F. Pizzati, et al. (2026) LongCoT: benchmarking long-horizon chain-of-thought reasoning. External Links: 2604.14140, [Link](#) Cited by: [§3.2](#), [Table 1](#), [§4](#).
- [24] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez (2023) MemGPT: towards llms as operating systems. External Links: 2310.08560, [Link](#) Cited by: [§4](#).
- [25] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang (2024) Training software engineering agents and verifiers with SWE-Gym. External Links: 2412.21139, [Link](#) Cited by: [§1](#), [§4](#).
- [26] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein (2023) Generative agents: interactive simulacra of human behavior. In ACM Symposium on User Interface Software and Technology, External Links: [Link](#) Cited by: [§4](#).
- [27] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun (2023) ChatDev: communicative agents for software development. External Links: 2307.07924, [Link](#) Cited by: [§4](#), [§4](#).
- [28] S. Rodionov (2026) Executable world models for ARC-AGI-3 in the era of coding agents. External Links: 2605.05138, [Document](#), [Link](#) Cited by: [§4](#).
- [29] E. Sarafian, G. Kaplun, R. Banner, D. Soudry, and B. Ginsburg (2026) Workspace optimization: how to train your agent. External Links: 2605.09650, [Document](#), [Link](#) Cited by: [§4](#), [§4](#).
- [30] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom (2023) Toolformer: language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, External Links: [Link](#) Cited by: [§4](#).
- [31] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao (2023) Reflexion: language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, External Links: [Link](#) Cited by: [§4](#).
- [32] C. Snell, J. Lee, K. Xu, and A. Kumar (2024) Scaling LLM test-time compute optimally can be more effective than scaling model parameters. External Links: 2408.03314, [Document](#), [Link](#) Cited by: [§1](#), [§4](#).
- [33] D. Tchuindjo, D. Shah, and O. Khattab (2026) OBLIQ-Bench: exposing overlooked bottlenecks in modern retrievers with latent and implicit queries. External Links: 2605.06235, [Link](#) Cited by: [§3.2](#), [Table 1](#), [§4](#).

- [34] A. M. Turing (1936) On computable numbers, with an application to the Entscheidungsproblem. Proceedings of the London Mathematical Society 42 (1), pp. 230–265. External Links: [Document](#) Cited by: [§1](#).
- [35] J. von Neumann (1945) First draft of a report on the EDVAC. Technical report Moore School of Electrical Engineering, University of Pennsylvania. Cited by: [§1](#).
- [36] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar (2023) Voyager: an open-ended embodied agent with large language models. External Links: 2305.16291, [Link](#) Cited by: [§4](#).
- [37] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji (2024) CodeAct: executable code actions elicit better llm agents. External Links: 2402.01030, [Link](#) Cited by: [§4](#), [§4](#).
- [38] X. Wang et al. (2024) OpenHands: an open platform for ai software developers as generalist agents. External Links: 2407.16741, [Link](#) Cited by: [§4](#).
- [39] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang (2023) AutoGen: enabling next-gen llm applications via multi-agent conversation. External Links: 2308.08155, [Link](#) Cited by: [§4](#), [§4](#).
- [40] Y. Xu, D. Lu, Z. Shen, J. Wang, Z. Wang, Y. Mao, C. Xiong, and T. Yu (2024) AgentTrek: agent trajectory synthesis via guiding replay with web tutorials. External Links: 2412.09605, [Link](#) Cited by: [§1](#), [§4](#).
- [41] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press (2024) SWE-agent: agent-computer interfaces enable automated software engineering. In Advances in Neural Information Processing Systems, External Links: 2405.15793, [Document](#), [Link](#) Cited by: [§4](#).
- [42] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao (2023) ReAct: synergizing reasoning and acting in language models. In International Conference on Learning Representations, External Links: [Link](#) Cited by: [§4](#).
- [43] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman (2022) STaR: bootstrapping reasoning with reasoning. In Advances in Neural Information Processing Systems, Vol. 35. External Links: [Link](#) Cited by: [§1](#), [§4](#).
- [44] A. L. Zhang, T. Kraska, and O. Khattab (2025) Recursive language models. External Links: 2512.24601, [Link](#) Cited by: [§1](#), [§1](#), [§2.3](#), [Table 1](#), [§4](#).
- [45] J. Zhang, T. Li, W. Jurayj, H. Zhan, B. V. Durme, and D. Khashabi (2026) Many-tier instruction hierarchy in llm agents. External Links: 2604.09443, [Link](#) Cited by: [Table 1](#), [Table 1](#).

Appendix A Example Out-of-Loop Experiments in the nanoGPT Speedrun

Each excerpt below is an experiment an agent created and ran outside the benchmark's training script during its nanoGPT run ([Section 3.3](#)), reproduced from the traces and trimmed for length.

Kimi K3 re-derived Newton–Schulz iteration coefficients with a global optimizer, checked bf16 rounding bit-exactly:

```
from scipy.optimize import differential_evolution

grid_in = np.concatenate([np.linspace(0.02, 0.05, 10),
np.linspace(0.05, 1.0, 190)])

grid_over = np.linspace(1.0, 1.3, 20)

def p_map(sig, a, b, c, iters=6):

    x = sig

    for _ in range(iters):

        x = a*x + b*x**3 + c*x**5

    return x

def objective(params):

    a, b, c = params

    dev = np.max(np.abs(p_map(grid_in, a, b, c) - 1.0))

    over = max(0.0, np.max(np.abs(p_map(grid_over, a, b, c))) - 1.15)

    return dev + 5.0*over

res = differential_evolution(objective,

[(1.0, 6.0), (-8.0, 0.0), (0.0, 5.0)],

maxiter=300, tol=1e-9, seed=0, polish=True)
```

DeepSeek V4 Pro built a calibrated toy of the training problem, with minibatch noise shaped by the true Kronecker Hessian and a natural-gradient oracle arm:

```
"""Calibrated toy: Kron-quadratic + CORRECT Kron-Hessian
```

```
minibatch noise. eps = Hl^{1/2} Z Hr^{1/2} / sqrt(n_eff).
```

```
Ideal preconditioner: Ql = Hl^{-1/2}, Qr = Hr^{-1/2}."""
```

```
G = Hl @ W @ Hr
```

```
eps = Hl12 @ torch.randn(p, q) @ Hr12 / (n_eff ** 0.5)
```

```
g = G + eps
```

```
elif opt == "natgrad":
```

```
El, Vl = torch.linalg.eigh(Hl)
```

```
Er, Vr = torch.linalg.eigh(Hr)
```

```
u = Vl @ (Vl.T @ d @ Vr /
```

```
(El[:, None]**0.5 * Er[None, :]**0.5)) @ Vr.T
```

GLM 5.3 debugged its SOAP implementation on CPU before any GPU screen:

```
torch.manual_seed(0)
```

```
for shape in [(768, 768), (3072, 768), (768, 3072)]:
```

```
p = torch.nn.Parameter((torch.randn(*shape) * 0.02).bfloat16())
```

```
opt = SOAP([p], lr=0.025)
```

```
for t in range(25):
```

```
p.grad = (torch.randn(*shape) * 0.01).to(torch.bfloat16)
```

```
opt.step()
```

```
if not torch.isfinite(p.data).all():

    print(shape, 'NaN at step', t+1)

    st = opt.state[p]

    print(' L finite', torch.isfinite(st['L']).all().item(),

        'v finite', torch.isfinite(st['v']).all().item())

    break
```

Appendix B Example Programmatic Orchestration

```
review = await rlm("Audit the implementation. Reply with concrete issues.",

    name="reviewer")

tests = await rlm("Run the test suite and classify failures.",

    name="tester")

children = await rlm.list_subagents()

await agent_message.send(

    "Also inspect error-handling edge cases.",

    receiver_role="child", receiver_name=review.name)
```

The explicit reply path is intentional: a child is a persistent concurrent session, not a stateless completion returned by rlm.

Appendix C LLM Usage Disclosure

Large language models were used to assist with code development, writing refinement, and formatting during the preparation of this manuscript. All scientific claims, experimental design, analysis, and intellectual contributions are solely the work of the authors.